INSTITUCIÓN UNIVERSITARIA ITM

Inteligencia de Negocios 190304015-1

Taller evaluativo 2

Proceso ETL con los datasets de Airbnb

Bogotá, Distrito Capital, Colombia

MongoDB → Python (pandas) → SQLite / XLSX

Autor

Samuel Cristobal Cuello Duque

Docente: Carlos Andrés Jaramillo Patiño

Repositorio: https://github.com/samuelcuello05/etl_airbnb

Medellín, 11 de Octubre de 2026

Contenido

1. Introducción
2. Descripción del dataset
3. Resumen del análisis exploratorio
4. Gráficas y hallazgos principales
5. Descripción de las transformaciones realizadas
6. Ejemplo del log generado
7. Conclusiones sobre la calidad y utilidad de los datos
8. Referencias

Anexo. Estructura del repositorio y ejecución

1. Introducción

Las plataformas de alquiler de corta estancia como Airbnb han transformado el mercado de vivienda y de turismo en las grandes ciudades. Bogotá, con más de diecinueve mil anuncios activos, es uno de los mercados más grandes de la región, y entender su oferta (dónde se ubica, a qué precio, qué tan disponible está y cómo evoluciona la demanda) es útil para hogares anfitriones, inversionistas, gremios turísticos y entidades distritales que regulan el uso del suelo y el registro de prestadores turísticos.

Sin embargo, los datos públicos de Airbnb no están listos para el análisis: llegan como texto (precios con símbolos de moneda, fechas y booleanos como cadenas), con campos anidados en formato JSON, columnas vacías, valores técnicos inválidos y variantes de escritura. Por eso es necesario un proceso de Extracción, Transformación y Carga (ETL) que los convierta en una base analítica confiable.

El objetivo de este trabajo fue implementar un proceso ETL automatizado en Python que (1) extrae los datos de Airbnb Bogotá desde una base de datos MongoDB local, (2) los explora para diagnosticar su calidad, (3) los limpia y transforma con reglas justificadas por ese diagnóstico, y (4) los carga en una base de datos SQLite y en archivos Excel, registrando cada paso en archivos de log. El proceso quedó implementado en tres clases (Extraccion, Transformacion y Carga), un módulo centralizado de logs y un notebook de análisis exploratorio.

2. Descripción del dataset

Los datos provienen de Inside Airbnb (insideairbnb.com), proyecto que recopila periódicamente la información pública de los anuncios de Airbnb. Se usó el scrape de Bogotá del 21-22 de junio de 2026, compuesto por tres archivos comprimidos. Siguiendo la guía, los archivos se cargaron primero en una base de datos MongoDB local (“airbnb_bogota_etl”) mediante el script src/cargar_mongo.py, que crea las colecciones Listings, Calendar y Reviews, les crea índices y valida que el número de documentos sea igual al de filas del archivo fuente. Todo el proceso posterior (EDA y ETL) lee exclusivamente de MongoDB.

Tabla 1. Colecciones cargadas en MongoDB

Colección	Archivo fuente	Documentos	Campos	Granularidad / contenido

Listings	listings.csv.gz	19.187	90	Un alojamiento: ubicación, tipo, capacidad, precio, anfitrión, amenities, disponibilidad y reseñas
Calendar	calendar.csv.gz	7.003.255	5	Un alojamiento × día (21-jun-2026 a 21-jun-2027): disponible, noches mínimas y máximas
Reviews	reviews.csv.gz	527.731	6	Una reseña: fecha, reseñador y comentario (oct-2010 a jun-2026)

Variables más relevantes de Listings: id y host_id (llaves), price (precio por noche en pesos colombianos, como texto "$284,294.01"), room_type (apartamento completo, habitación privada, compartida u hotel), neighbourhood_cleansed (una de las 20 localidades de Bogotá), latitude/longitude, accommodates, bedrooms, beds, bathrooms, minimum_nights, maximum_nights, availability_365, number_of_reviews, review_scores_* (calificaciones de 1 a 5), amenities (lista JSON de servicios), price_quote_raw (cotización en JSON) y 16 campos host_* con la información del anfitrión. Una particularidad de este scrape es que Calendar ya no incluye el precio diario, por lo que el precio para análisis económicos debe tomarse de Listings.

3. Resumen del análisis exploratorio

El análisis exploratorio se desarrolló en notebooks/exploracion_airbnb.ipynb. Para cada colección se revisaron las primeras filas, la cantidad de registros y columnas, los tipos de datos y la descripción estadística de las variables principales; luego se evaluó la calidad de los datos y se estudiaron distribuciones, relaciones y tendencias. Los resultados de calidad se resumen a continuación.

Tabla 2. Diagnóstico de calidad de los datos

Dimensión	Hallazgo	Implicación
Nulos	13 columnas de Listings 100 % vacías (host_since, host_response_rate, neighbourhood, calendar_updated, instant_bookable…); 22 % sin calificaciones (alojamientos sin reseñas); 195 precios nulos; bedrooms 15 %; bathrooms 6 %; 191 comentarios nulos	Eliminar columnas vacías; no imputar calificaciones; imputar atributos físicos y precio con reglas
Duplicados	0 duplicados exactos y 0 por llave (id; listing_id+date; id de reseña). 0 registros huérfanos en Calendar y Reviews	No se elimina nada, pero el ETL conserva el control preventivo
Atípicos	price: mediana ≈161 mil COP, P99 ≈1,33 M y máximo de 448 M COP/noche; ~3,5 % atípicos por IQR sobre log(precio). maximum_nights con el valor técnico 2.147.483.647. minimum_nights de 30 (17 %) y availability_365 = 365 (18 %) son legítimos	Marcar y winsorizar el precio sin borrar; acotar noches a 1.125; conservar el resto con banderas
Formato	Precio como texto con $ y ,; fechas y booleanos como texto; HTML en ~69 mil comentarios; variantes "Bogota/Bogotá"; localidades sin tilde	Normalizar moneda, fechas (YYYY-MM-DD), booleanos (1/0) y texto
Estructura	amenities y price_quote_raw en JSON; 16 campos del host repetidos por anuncio (8.335 hosts / 19.187 anuncios); Calendar de 7 M filas	Desanidar en tablas propias y resumir el calendario por mes/semana

En síntesis, los datos son consistentes en sus llaves y entre colecciones (no hay duplicados ni registros huérfanos), pero presentan problemas de formato, valores extremos y estructuras anidadas que impiden su uso directo en una base de datos relacional o en una herramienta de inteligencia de negocios.

4. Gráficas y hallazgos principales

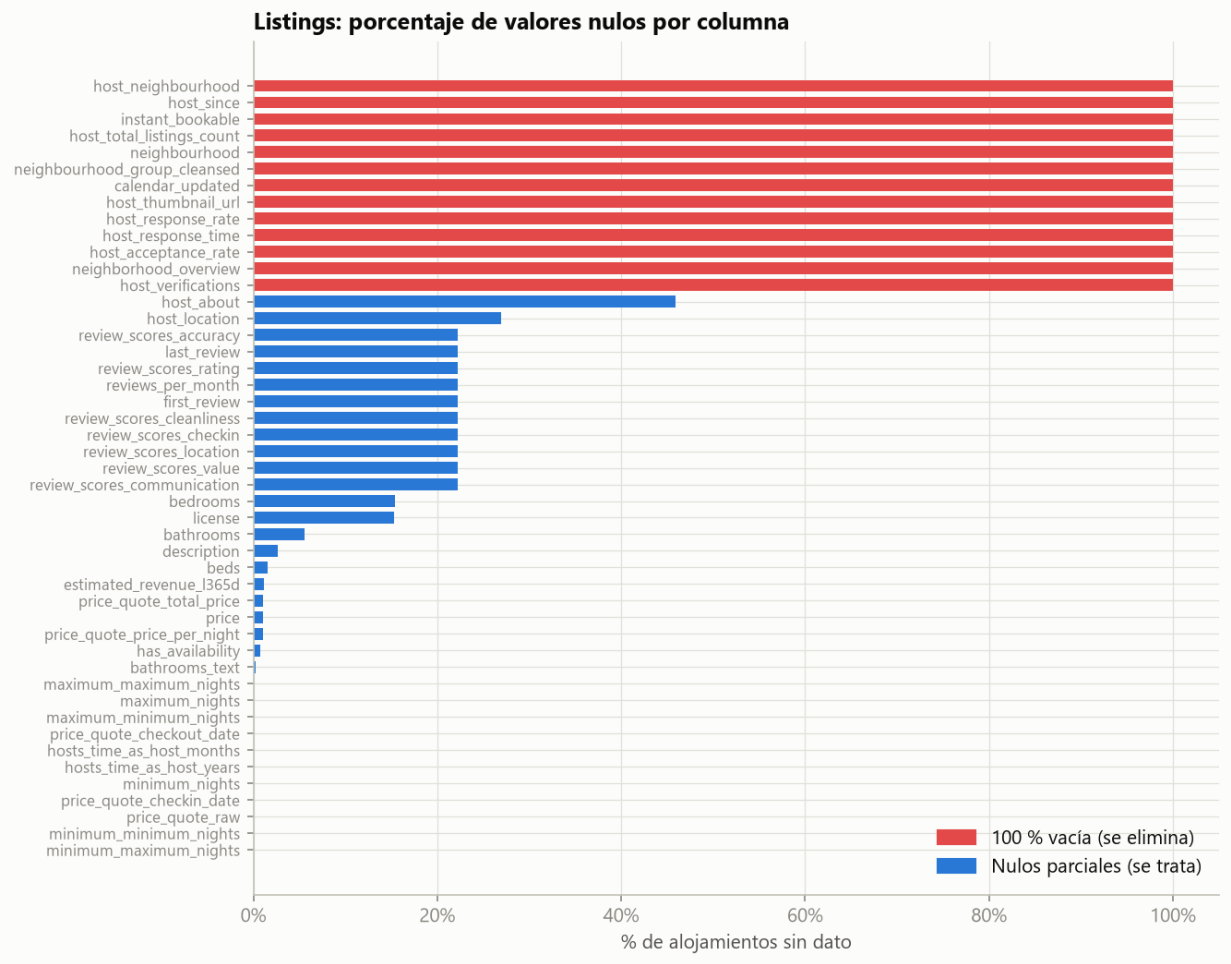


Figura 1. Porcentaje de valores nulos por columna en Listings

Interpretación: las 13 columnas en rojo están completamente vacías en este scrape y se eliminan. El bloque de ~22 % corresponde a variables de reseñas de alojamientos que nunca han sido reseñados: no son datos perdidos sino ausencia de calificación, por lo que no se imputan.

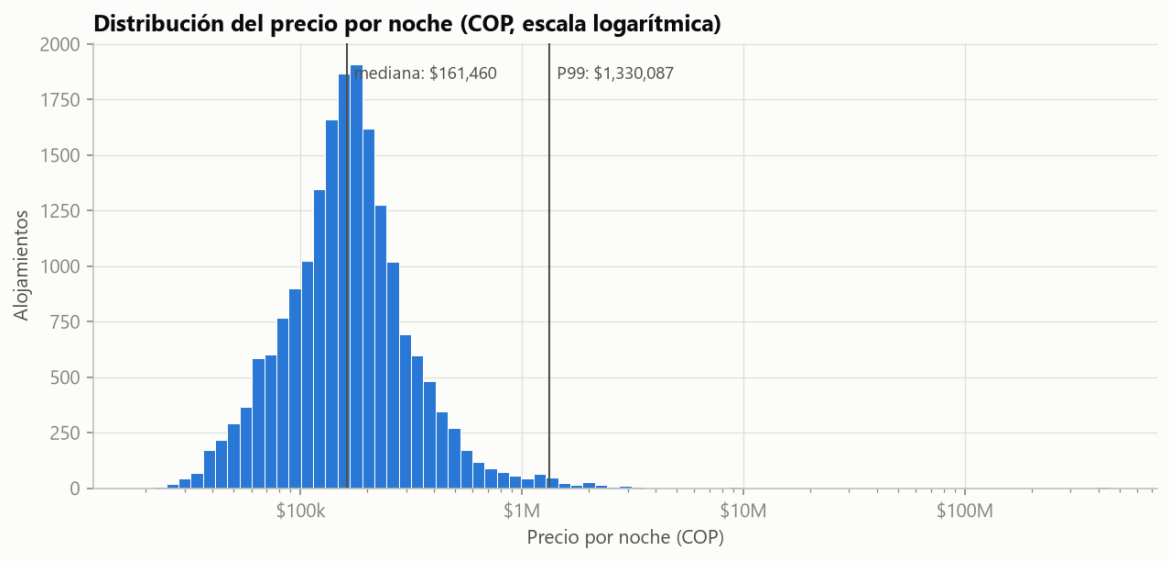


Figura 2. Distribución del precio por noche (escala logarítmica)

Interpretación: el precio es muy asimétrico: la mitad de los alojamientos cuesta menos de ~161 mil COP por noche, pero existen valores de hasta 448 millones que son errores de captura. En escala logarítmica la distribución es casi normal; por eso los atípicos se identifican con el IQR del logaritmo y el precio se winsoriza a P1–P99.

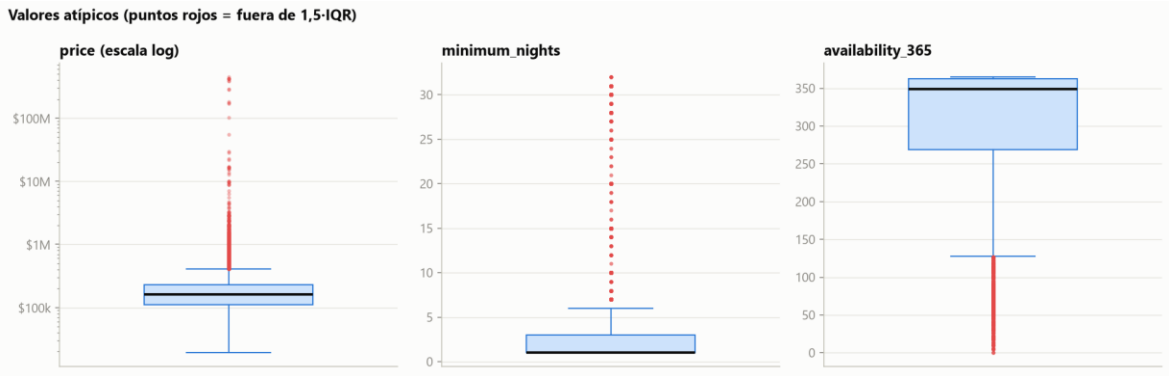


Figura 3. Valores atípicos en price, minimum_nights y availability_365

Interpretación: solo en el precio los atípicos son errores evidentes. En minimum_nights los “atípicos” son estadías mensuales (30 noches) y en availability_365 son alojamientos muy reservados: ambos describen segmentos reales del mercado y se conservan.

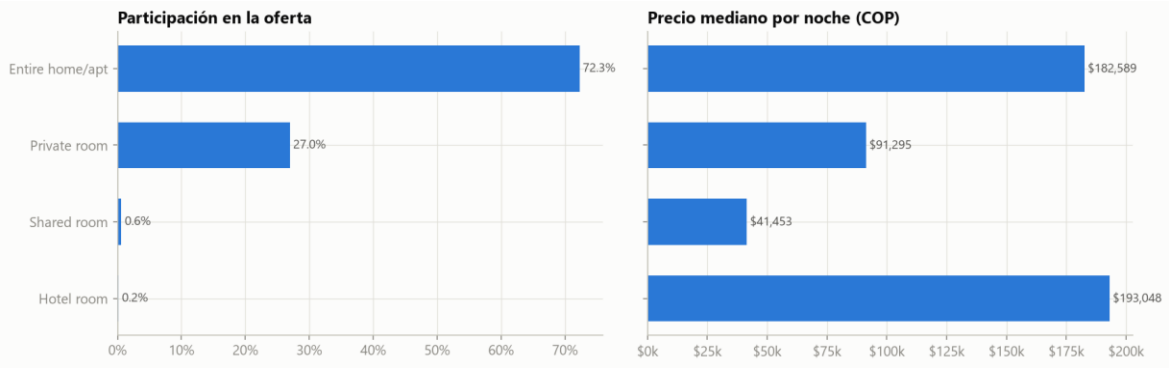


Figura 4. Participación y precio mediano por tipo de alojamiento

Interpretación: el 72 % de la oferta son apartamentos completos, que cuestan en mediana el doble que una habitación privada. Como el tipo de alojamiento determina el precio, la imputación de precios faltantes se hace por localidad y tipo de habitación.

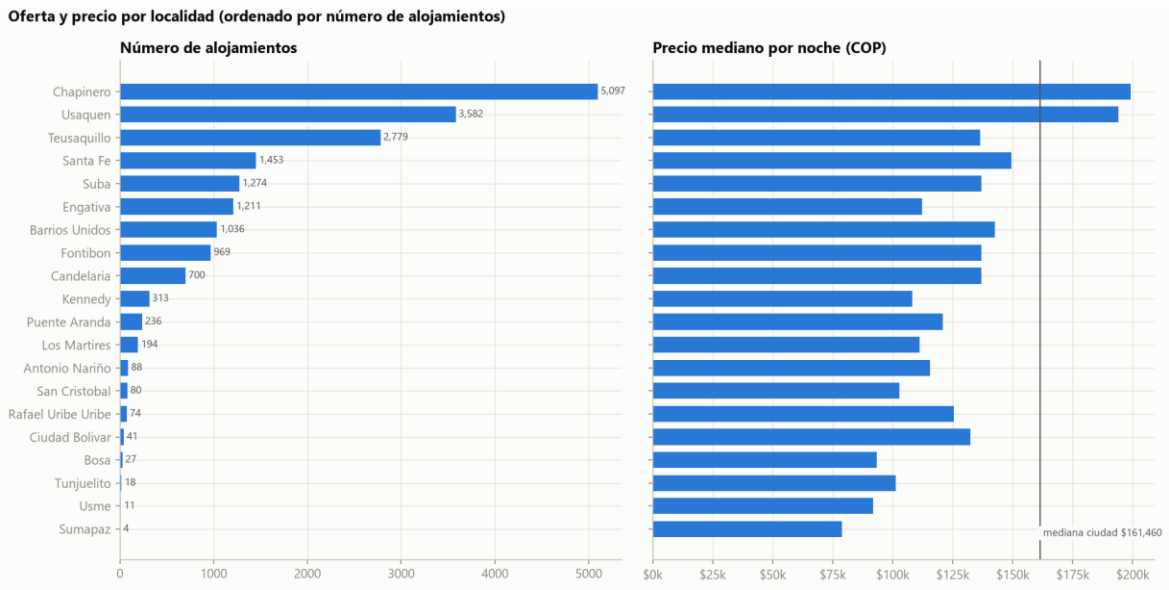


Figura 5. Oferta y precio mediano por localidad

Interpretación: Chapinero, Usaquén y Teusaquillo concentran el 60 % de la oferta, y Chapinero y Usaquén son las localidades más caras (~195-200 mil COP). Las localidades del sur tienen precios menores y muy pocos anuncios, por lo que sus medianas son menos estables.

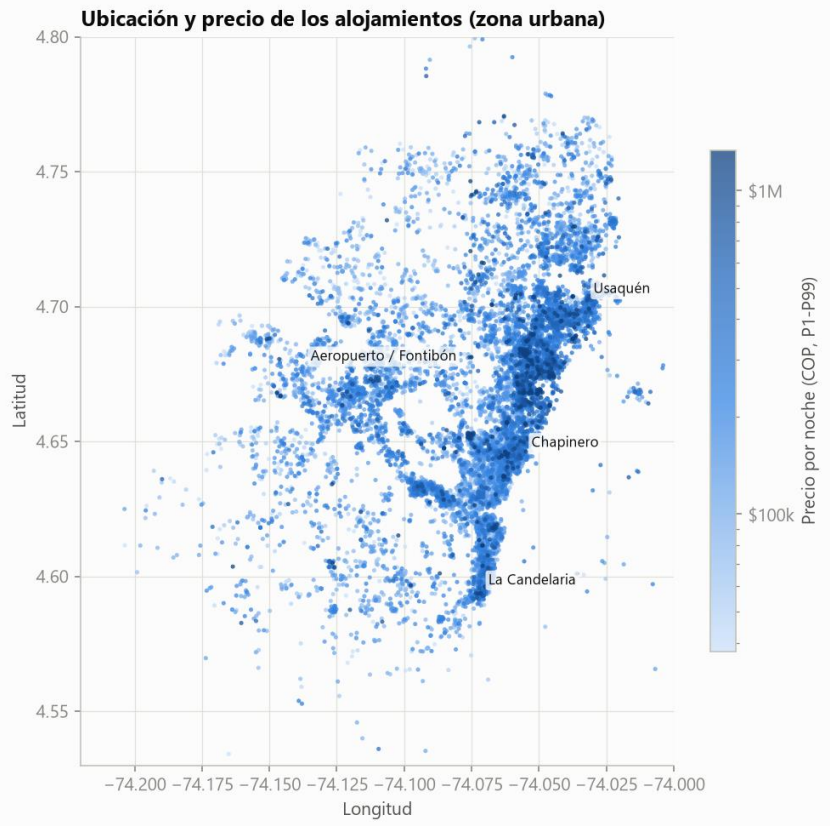


Figura 6. Ubicación y precio de los alojamientos

Interpretación: se observa un corredor nororiental (Chapinero–Usaquén) de alta densidad y precio, un núcleo turístico en el centro histórico y otro cerca del aeropuerto. Todas las coordenadas están dentro de Bogotá.

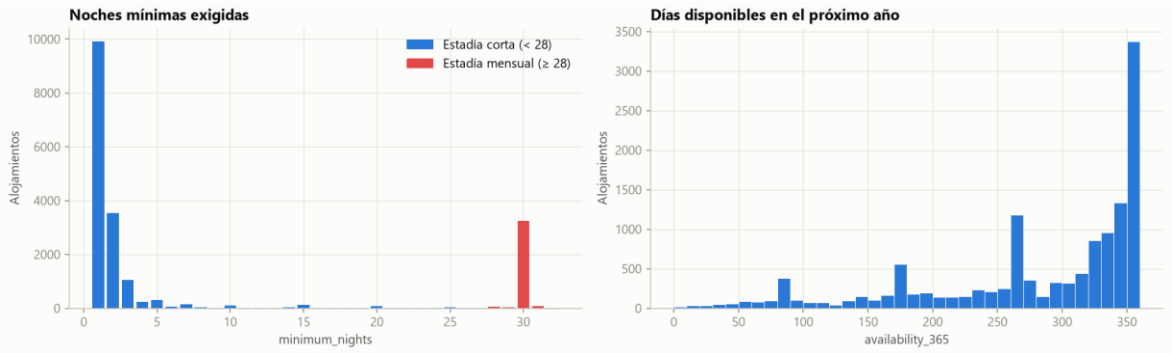


Figura 7. Noches mínimas y días disponibles en el próximo año

Interpretación: coexisten dos mercados: el turístico de estadía corta (52 % acepta una noche) y uno de arriendo mensual (~18 % exige 28 noches o más). El 18 % de los anuncios está disponible los 365 días, señal de posibles anuncios inactivos; se marca con la bandera disponible_todo_el_anio.

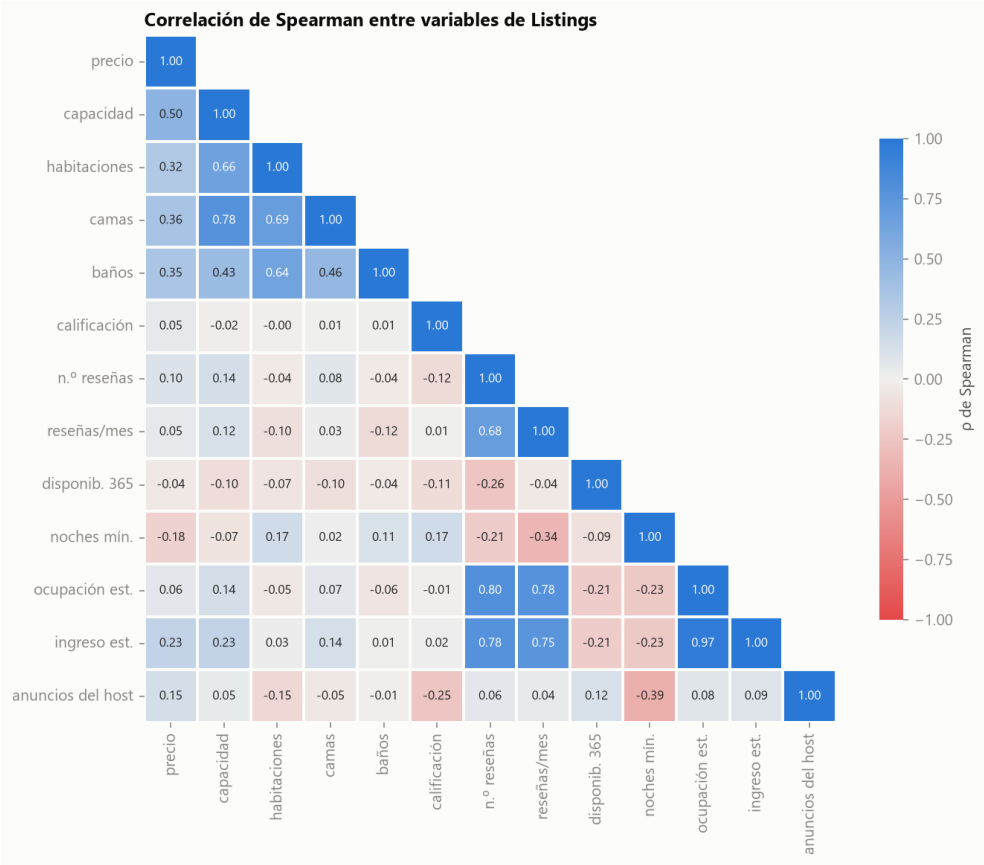


Figura 8. Correlación de Spearman entre variables de Listings

Interpretación: el precio depende sobre todo del tamaño del alojamiento (capacidad ρ≈0,50) y casi nada de la calificación (ρ≈0,05). Reseñas, ocupación e ingreso estimados están muy correlacionados (ρ 0,7-0,97) porque Inside Airbnb estima la ocupación a partir de las reseñas, así que no son evidencia independiente de demanda.

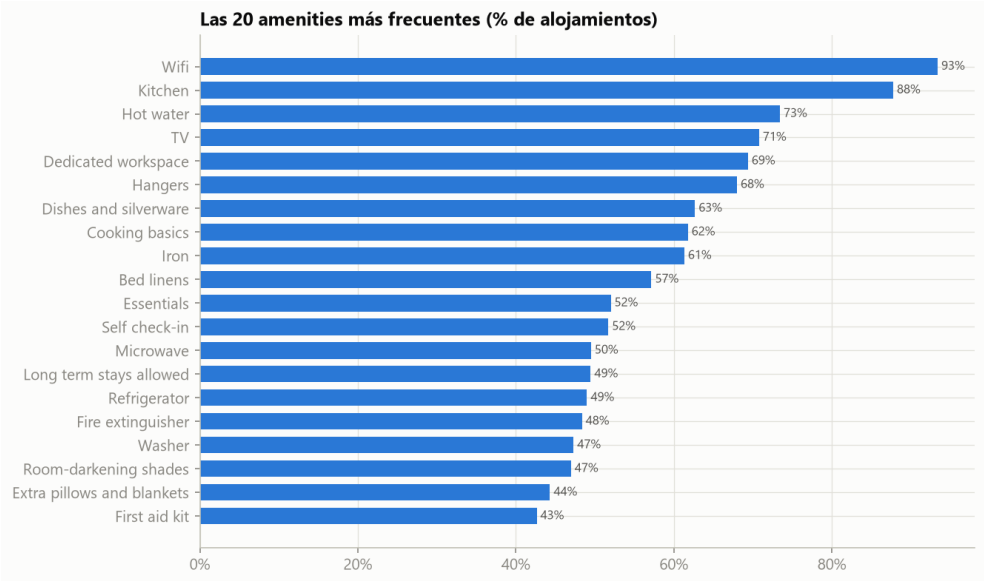


Figura 9. Las 20 amenities más frecuentes

Interpretación: wifi y cocina son prácticamente estándar. El campo amenities tiene más de 3.600 valores distintos dentro de una lista JSON, por lo que se desanida en una tabla larga y en columnas indicadoras para las amenities clave.

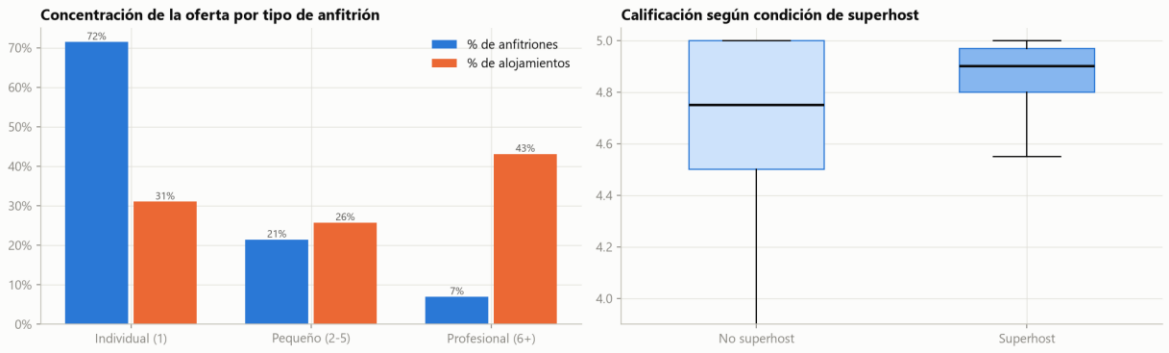


Figura 10. Concentración de la oferta por tipo de anfitrión y calificación de superhosts

Interpretación: el 72 % de los anfitriones tiene un solo anuncio, pero el 7 % con seis o más controla el 43 % de la oferta: el mercado está profesionalizado. Los superhosts tienen mejor calificación (4,90 vs 4,75). La información del host se separa en una dimensión propia para no repetirla en cada anuncio.

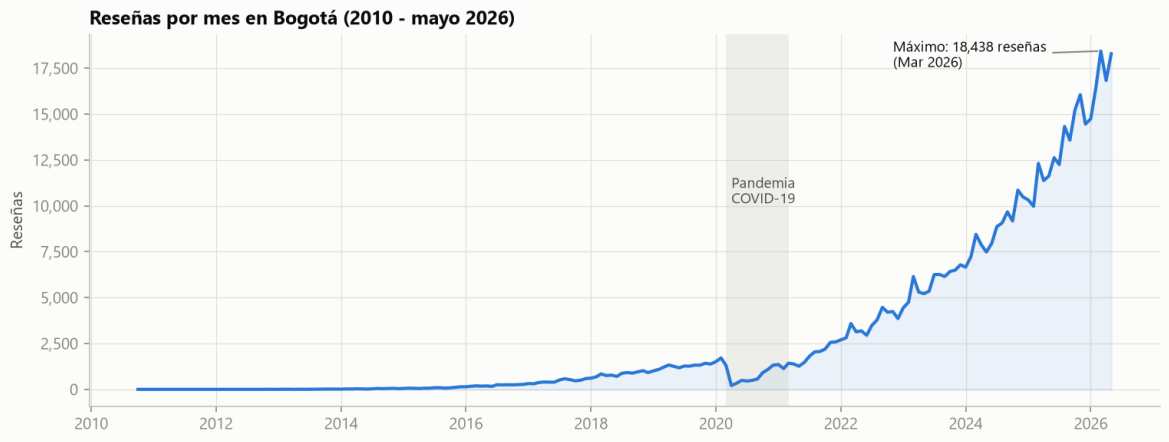


Figura 11. Reseñas por mes en Bogotá (2010 – mayo 2026)

Interpretación: la demanda creció de forma sostenida desde 2015, cayó 31 % en 2020 por la pandemia y se recuperó rápidamente: 2021 ya superó a 2019 (+41 %) y 2025 cerró con más de 154 mil reseñas, diez veces el nivel de 2019; el máximo mensual fue marzo de 2026. Sobre la tendencia se observan oscilaciones estacionales.

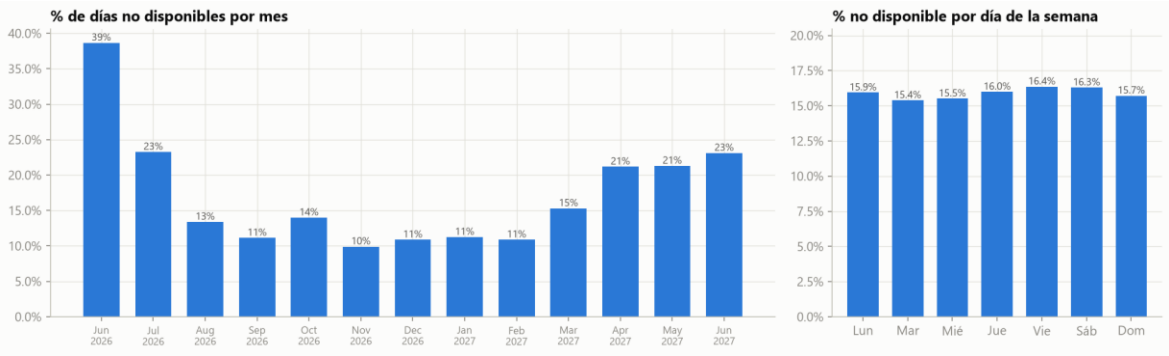


Figura 12. Porcentaje de días no disponibles por mes y por día de la semana

Interpretación: el calendario es prospectivo: la no disponibilidad es alta en las fechas cercanas (reservas ya hechas para junio-julio de 2026) y baja en meses lejanos. Como "no disponible" mezcla reservas y bloqueos del anfitrión, se nombra no_disponible (no "ocupado") y se analiza por mes y semana.

5. Descripción de las transformaciones realizadas

Las transformaciones se implementaron en la clase Transformacion (src/transformacion.py). Cada método tiene un docstring que explica por qué se aplica, y cada paso registra en el log las filas antes y después y las advertencias encontradas; además se guarda una tabla de auditoría en SQLite y Excel. La siguiente tabla relaciona cada hallazgo del EDA con la decisión tomada.

Tabla 3. Hallazgos del EDA y transformaciones aplicadas

Hallazgo	Transformación (método)	Justificación
Columnas 100 % vacías y URLs	Eliminación (_eliminar_columnas_vacias)	No se pueden imputar y no aportan al análisis
price como texto "$284,294.01"	Quitar $ y , y convertir a float COP (normalizar_precio)	Requisito para cálculos y agregaciones
195 precios nulos	Mediana por localidad × tipo de habitación + precio_imputado	Grupo más parecido según el EDA; la bandera permite excluirlos
Precios extremos (máx. 448 M)	precio_outlier (IQR en log) y price_ajustado (P1–P99)	No se borran anuncios reales de lujo, pero se evita distorsionar promedios
Segmentación de precio	categoria_precio: Económico < 100k, Medio 100-200k, Alto 200-400k, Premium 400k-1M, Lujo ≥ 1M	Rangos basados en la distribución observada
Fechas y booleanos como texto	Fechas a YYYY-MM-DD; año, mes, día, trimestre, día de la semana, semana ISO y fin de semana; 't'/'f' → 1/0	Formato estándar ISO (recomendado para SQLite) y análisis por periodos
maximum_nights = 2.147.483.647	Acotar a 1.125 (acotar_noches)	Valor técnico, no una estadía real
Nulos de reseñas, baños, habitaciones	tiene_resenas; reviews_per_month = 0; baños desde bathrooms_text; habitaciones/camas por capacidad	Se imputa solo lo que tiene una regla defendible

Textos inconsistentes	canonizar_texto (Bogota → Bogotá), localidad con nombre oficial, limpiar_texto	Evita que un GROUP BY divida la misma categoría
amenities en JSON	Tabla listing_amenities, amenities_resumen, amenities_count y 14 columnas 0/1	Permite filtrar y agregar en SQL
Datos del host repetidos	Dimensión hosts con tipo_anfitrion; listings conserva host_id	Normalización: elimina redundancia
price_quote_raw en JSON	Se extrae y valida la moneda (100 % COP); cargos vacíos se descartan	Asegura que todos los precios son comparables
Calendar de 7 M filas	disponible/no_disponible, variables de fecha, calendar_mensual (con ingreso estimado) y calendar_semanal	Excel admite 1.048.576 filas; análisis de temporada
Comentarios con HTML y nulos	limpiar_texto, “Sin comentario”, longitud_comentario, comentario_corto, reviews_mensual	Texto limpio para análisis futuros y tendencia mensual
Duplicados / huérfanos	eliminar_duplicados e integridad referencial	Control preventivo en cada carga

Registro de transformaciones (auditoría real de la ejecución)

Tabla 4. Tabla de auditoría generada por el ETL (filas antes y después de cada paso)

Tabla	Paso	Antes	Después	Detalle
listings	Eliminar duplicados	19.187	19.187	duplicados por ['id'] = 0
listings	Eliminar columnas vacías/irrelevantes	19.187	19.187	13 vacías + 9 irrelevantes; quedan 68 columnas
listings	Desanidar price_quote_raw (JSON)	19.187	19.187	monedas={'COP': 19187}
listings	Normalizar precio ($ y , -> número COP)	19.187	19.187	imputados=195, outliers=667, winsorizado P1=37,610 P99=1,318,301
listings	Categorizar precio por rangos	19.187	19.187	{'Medio': 8680, 'Alto': 4946, 'Económico': 4062, 'Premium': 1192, 'Lujo': 307}
listings	Booleanos t/f -> 1/0	19.187	19.187	host_is_superhost, host_has_profile_pic, host_identity_verified, has_availability
listings	Fechas -> YYYY-MM-DD	19.187	19.187	last_scraped, first_review, last_review, price_quote_*_date
listings	Tratar valores nulos	19.187	19.187	celdas nulas 98,970 -> 87,996 (review_scores sin reseñas se conservan nulos: 4,265 alojamientos)
listings	Derivar baños desde bathrooms_text	19.187	19.187	bathrooms nulos 1066 -> 0
listings	Estandarizar textos y localidades	19.187	19.187	20 localidades oficiales
listing_amenities	Desanidar amenities (JSON -> tabla larga)	19.187	505.595	3,687 amenities distintas
hosts	Desanidar información del host (dimensión)	19.187	8.335	8,335 anfitriones únicos para 19,187 alojamientos
calendar	Eliminar duplicados	7.003.255	7.003.255	duplicados por ['listing_id', 'date'] = 0
calendar	Integridad referencial con listings	7.003.255	7.003.255	

calendar	Fecha YYYY-MM-DD + año/mes/día/trimestre/semana	7.003.255	7.003.255	rango 2026-06-21 a 2027-06-21
calendar_mensual	Agrupar calendario por alojamiento y mes	7.003.255	249.431	
calendar_semanal	Agrupar calendario por semana (ciudad)	7.003.255	54	
reviews	Eliminar duplicados	527.731	527.731	duplicados por ['id'] = 0
reviews	Integridad referencial con listings	527.731	527.731	
reviews	Limpiar comentarios (HTML, espacios, nulos)	527.731	527.731	nulos/vacíos=191, cortos=5281
reviews	Fecha YYYY-MM-DD + año/mes/día/trimestre	527.731	527.731	rango 2010-10-06 a 2026-06-21
reviews_mensual	Agrupar reseñas por mes	527.731	182	

Efecto sobre el precio: el promedio pasó de 442.995 COP (distorsionado por el máximo de 448.176.567 COP) a 205.806 COP con el precio ajustado; se imputaron 195 precios y se marcaron 667 atípicos. Distribución por categoría: Económico 4.062 (21%), Medio 8.680 (45%), Alto 4.946 (26%), Premium 1.192 (6%), Lujo 307 (2%).

Carga y verificación

La clase Carga (src/carga.py) crea una base SQLite nueva en cada ejecución, inserta las tablas por bloques (para no agotar la memoria con los 7 millones de filas del calendario), crea índices y exporta las tablas a Excel con xlsxwriter en modo de memoria constante. Finalmente verifica con SELECT COUNT(*) que cada tabla tenga las mismas filas que su DataFrame, que el precio quedó como REAL y que todas las fechas cumplen el formato YYYY-MM-DD, y relee cada hoja de Excel para comparar el número de filas.

Tabla 5. Tablas cargadas en SQLite (data/processed/airbnb_bogota.db)

Tabla	Registros	Columnas	Exportada a Excel
listings	19.187	77	airbnb_bogota_resumen.xlsx
hosts	8.335	17	airbnb_bogota_resumen.xlsx
listing_amenities	505.595	2	airbnb_bogota_resumen.xlsx
amenities_resumen	3.687	3	airbnb_bogota_resumen.xlsx
calendar	7.003.255	13	Solo SQLite (supera el límite de Excel)
calendar_mensual	249.431	13	airbnb_bogota_resumen.xlsx
calendar_semanal	54	5	airbnb_bogota_resumen.xlsx
reviews	527.731	16	airbnb_bogota_reviews.xlsx
reviews_mensual	182	6	airbnb_bogota_resumen.xlsx
auditoria_transformaciones	22	5	airbnb_bogota_resumen.xlsx

6. Ejemplo del log generado

El módulo src/logger.py centraliza los logs: cada ejecución genera un archivo logs/log_YYYYMMDD_HHMM.txt, con fecha, hora, nivel (INFO, WARNING, ERROR), componente y descripción del evento; los mismos mensajes se muestran en consola. A continuación, extractos reales de la ejecución (log_20261006_2146.txt).

```
2026-10-06 21:46:50 | INFO     | etl.main | ########## INICIO DEL PROCESO ETL AIRBNB BOGOTÁ ##########
2026-10-06 21:46:50 | INFO     | etl.extraccion | Conexión exitosa a MongoDB mongodb://localhost:27017/ (versión
8.3.8), base de datos 'airbnb_bogota_etl'
2026-10-06 21:46:51 | INFO     | etl.extraccion | Colección 'Listings': 19,187 registros y 90 columnas extraídos (1.0s)
2026-10-06 21:47:31 | INFO     | etl.transformacion | [listings] Columnas 100% nulas eliminadas (13):
['neighborhood_overview', 'host_since', 'host_response_time'...
2026-10-06 21:47:35 | WARNING  | etl.transformacion | [listings] 195 alojamientos sin precio; se imputan con la mediana
por localidad y tipo de habitación
2026-10-06 21:47:35 | WARNING  | etl.transformacion | [listings] 667 precios atípicos (fuera de 36460 - 689680 COP)
marcados; precio máximo encontrado 448176567 COP
2026-10-06 21:47:35 | INFO     | etl.transformacion | [listings] Normalizar precio ($ y , -> número COP) | filas:
19,187 -> 19,187 | imputados=195, outliers=667, ...
2026-10-06 21:47:35 | WARNING  | etl.transformacion | [listings] 'maximum_nights': 6 valores > 1125 acotados a 1125
2026-10-06 21:47:38 | INFO     | etl.transformacion | [listing_amenities] Desanidar amenities (JSON -> tabla larga) |
filas: 19,187 -> 505,595 | 3,687 amenities d...
2026-10-06 21:47:38 | INFO     | etl.transformacion | [hosts] Desanidar información del host (dimensión) | filas:
19,187 -> 8,335 | 8,335 anfitriones únicos para ...
2026-10-06 21:47:46 | WARNING  | etl.transformacion | [reviews] 191 comentarios nulos rellenados con 'Sin comentario'
2026-10-06 21:47:47 | INFO     | etl.transformacion | Tabla lista para carga: calendar                   7,003,255
filas x 13 columnas
2026-10-06 21:48:02 | INFO     | etl.carga | SQLite: tabla 'calendar' cargada con 7,003,255 registros (14.2s)
2026-10-06 21:48:35 | WARNING  | etl.carga | XLSX: 'calendar' tiene 7,003,255 filas y supera el límite de Excel
(1,048,575); no se exporta a XLSX (queda completa ...
2026-10-06 21:49:52 | INFO     | etl.carga | Verificación SQLite OK  listings                   19,187 = 19,187
2026-10-06 21:49:52 | INFO     | etl.carga | Verificación SQLite OK  calendar                   7,003,255 = 7,003,255
2026-10-06 21:49:54 | INFO     | etl.carga | Verificación de tipos OK: price es REAL y todas las fechas son YYYY-MM-DD
2026-10-06 21:49:54 | INFO     | etl.carga | Carga verificada correctamente: todos los registros coinciden
2026-10-06 21:49:54 | INFO     | etl.main | ########## FIN DEL PROCESO ETL (OK) en 183.6s ##########
```

Se observa el uso de los tres niveles: INFO para el avance normal (conexión, registros extraídos, filas antes y después), WARNING para situaciones que requirieron una decisión (precios imputados, atípicos, valores acotados, tabla que no cabe en Excel) y ERROR, que se registra con su traza completa cuando un paso falla (por ejemplo, si MongoDB no está disponible o si la verificación de la carga encuentra diferencias).

Log de la carga inicial de MongoDB (log_20261006_2119.txt):

```
2026-10-06 21:19:40 | INFO     | etl.cargar_mongo | Validación OK 'Listings': 19,187 filas en CSV = 19,187 documentos
en MongoDB (2.7s)
2026-10-06 21:21:36 | INFO     | etl.cargar_mongo | Validación OK 'Calendar': 7,003,255 filas en CSV = 7,003,255
documentos en MongoDB (116.3s)
2026-10-06 21:21:48 | INFO     | etl.cargar_mongo | Validación OK 'Reviews': 527,731 filas en CSV = 527,731 documentos
en MongoDB (11.8s)
2026-10-06 21:21:48 | INFO     | etl.cargar_mongo | Colecciones disponibles en 'airbnb_bogota_etl': ['Calendar',
'Listings', 'Reviews']
```

7. Conclusiones sobre la calidad y utilidad de los datos

- **Calidad estructural alta, calidad de formato baja.** Las tres colecciones son íntegras: no hay duplicados, todas las fechas y llaves son válidas y no existen registros huérfanos. Los problemas están en el formato (moneda, fechas, booleanos y texto como cadenas), en valores extremos o técnicos (precio de 448 millones, noches máximas de 2.147.483.647) y en estructuras anidadas. Todos son corregibles con reglas explícitas, como se hizo en el ETL.
- **Pérdida de información en la fuente.** Trece columnas llegaron completamente vacías (tiempo de respuesta del host, fecha de registro, reserva inmediata…) y el calendario ya no publica precios diarios. Esto limita análisis de calidad del servicio y de precios dinámicos por temporada; el ingreso estimado del calendario usa el precio base del anuncio y debe interpretarse como aproximación.
- **La disponibilidad no es ocupación.** available = 'f' mezcla reservas y bloqueos del anfitrión, y el 18 % de los anuncios está disponible todo el año. Para medir demanda real son más confiables las reseñas (con la salvedad de que no todo huésped reseña) que el calendario; por eso se nombró la variable no_disponible y se conservaron banderas en lugar de eliminar registros.
- **Los datos son útiles para decisiones de negocio.** Tras el ETL es posible responder con SQL o Excel preguntas como: ¿qué localidades y tipos de alojamiento tienen mayor precio?, ¿qué tan concentrada está la oferta en anfitriones profesionales (7 % de hosts con 43 % de los anuncios)?, ¿cómo evoluciona la demanda (crecimiento de diez veces frente a 2019)?, ¿qué amenities diferencian a los anuncios más caros? La base queda lista para un tablero en Power BI o para el proyecto final.
- **Decisiones conservadoras y trazables.** Se prefirió marcar (precio_outlier, precio_imputado, tiene_resenas, disponible_todo_el_anio, comentario_vacio) antes que borrar, de modo que el analista pueda incluir o excluir esos casos según la pregunta. Cada decisión quedó registrada en el log y en la tabla de auditoría.
- **Recomendaciones.** Automatizar la descarga periódica de nuevos scrapes para construir series históricas de precio, cruzar la localidad con datos del Distrito (estratos, Registro Nacional de Turismo) y, como valor agregado, migrar la carga a PostgreSQL para el proyecto final.

8. Referencias

Inside Airbnb. (2026). Get the Data – Bogotá, Distrito Capital, Colombia [Conjunto de datos]. https://insideairbnb.com/get-the-data/

Inside Airbnb. (2026). Data Dictionary. https://insideairbnb.com/data-assumptions/

MongoDB, Inc. (2026). PyMongo documentation. https://pymongo.readthedocs.io/

The pandas development team. (2026). pandas documentation (v3.0). https://pandas.pydata.org/docs/

SQLite. (2026). Datatypes In SQLite (fechas como TEXT ISO-8601). https://www.sqlite.org/datatype3.html

Python Software Foundation. (2026). logging — Logging facility for Python. https://docs.python.org/3/library/logging.html

McNamara, J. (2026). XlsxWriter documentation. https://xlsxwriter.readthedocs.io/

Waskom, M. (2021). seaborn: statistical data visualization. Journal of Open Source Software, 6(60), 3021.

Kimball, R., & Ross, M. (2013). The Data Warehouse Toolkit (3.ª ed.). Wiley.

Anexo. Estructura del repositorio y ejecución

```
etl_airbnb/
├── data/raw/            listings.csv.gz, calendar.csv.gz, reviews.csv.gz (fuente para poblar MongoDB)
├── data/processed/      airbnb_bogota.db (SQLite) y archivos .xlsx generados
├── src/                 config.py, logger.py, cargar_mongo.py, extraccion.py, transformacion.py,
│                        carga.py, main.py
├── notebooks/           exploracion_airbnb.ipynb
├── reports/             informe en PDF y figuras del EDA
├── logs/                log_YYYYMMDD_HHMM.txt (uno por ejecución)
├── README.md
└── requirements.txt

python -m venv .venv  &&  .venv\Scripts\activate
pip install -r requirements.txt
python src/cargar_mongo.py     # 1 vez: crea las colecciones en MongoDB
python src/main.py             # ETL: MongoDB -> SQLite + XLSX
```